

Paradigmas de Programación

Entrega 2

Desarrollar un juego en C#.Net (utilizando el engine provisto) que incluya lo solicitado para la entrega anterior:

1. Un menú principal.
2. Un nivel jugable con movimiento del jugador.
3. Que se pueda ganar y que muestre una pantalla de victoria.
4. Que se pueda perder y que muestre una pantalla de derrota.
5. Usar el Singleton tradicional en un GameManager que administre los datos globales de la aplicación.
6. Objetos que usen animaciones.
7. Detección de colisiones entre objetos.
8. Herencia.
9. Factory (por ejemplo, de enemigos, armas, ítems, etc).
10. Crear y usar eventos en situaciones que lo ameriten (ejemplos: pérdida de vida, muerte de enemigos, etc).
11. Implementar interfaces diferentes que tengan sentido en el juego.

Y agregar:

12. Implementación de un Pool GENÉRICO (que use generics para que el pool pueda crear cualquier tipo de objeto pooleable). Debe ser usado en alguna de las clases.
13. Revisar todo el código para verificar que se cumplan los principios SOLID y STUPID.

Rúbrica

Criterio	0%	25%	50%	75%	100%	Puntaje total
Herencia	No utiliza herencia	Intenta usar herencia pero con errores de diseño o sin aportar reutilización real	Usa herencia correctamente en al menos un caso con relación “es un” bien aplicada	Usa múltiples casos de herencia coherentes con reutilización clara	Jerarquías bien diseñadas con polimorfismo (virtual/abstract) y sin problemas evidentes, sin encontrarse oportunidades de mejora	5
Interfaces	No utiliza interfaces	Define interfaces pero no las usa correctamente o están acopladas a implementaciones	Usa al menos una interfaz correctamente desacoplando comportamiento	Uso consistente de varias interfaces favoreciendo extensibilidad	Interfaces bien diseñadas y segregadas que permiten desacople y fácil extensión, sin encontrarse oportunidades de mejora	10
Singleton	No utiliza Singleton cuando sería útil	Implementación incorrecta (no garantiza instancia única o mal acceso global)	Implementación correcta básica con instancia única	Uso adecuado en casos necesarios evitando abuso	Uso criterioso sin acoplamiento excesivo e integrado correctamente al diseño	10
Factory	No utiliza Factory	Implementación incorrecta o innecesaria (uso de new directo en múltiples lugares)	Usa Factory en al menos un caso centralizando creación	Uso extendido para múltiples objetos mejorando mantenibilidad	Diseño flexible que permite agregar tipos sin modificar código, sin encontrarse oportunidades de mejora	5
Eventos	No utiliza eventos	Uso incorrecto o innecesario con lógica acoplada	Usa eventos en al menos un caso para comunicación básica	Uso consistente desacoplando sistemas (UI, lógica, etc.)	Arquitectura basada en eventos con buena organización y responsabilidades claras	5
Diferenciación	Proyecto igual al visto en clase	Cambios mínimos (solo estética o detalles)	Introduce alguna mecánica nueva pero sigue muy basado en ejemplos	Proyecto claramente distinto con decisiones propias	Proyecto original con enfoque creativo y exploración más allá de lo visto en la cursada	10
Loop de juego	No hay loop completo o el juego no es jugable	Loop funcional básico (pantalla de inicio y juego)	Loop completo con inicio juego fin, pero no reinicia o reinicia con errores.	Loop completo con inicio juego fin y reinicio	Loop robusto con estados bien definidos (inicio juego pausa victoria derrota reinicio) y buena transición entre ellos	10
Animaciones y colisiones	No hay animaciones o colisiones	Implementación muy básica o con errores evidentes ya sea en colisiones o animaciones	Animaciones y colisiones funcionando correctamente en casos simples	Ambos sistemas bien implementados y utilizados en varias situaciones	Uso sólido y consistente con integración al gameplay (feedback visual y físicas bien resueltas)	5
Pool	No se implementa Pool	Se observa un intento de implementación de un pool, pero no se logra hacerlo correctamente.	Se implementa un Pool no genérico, y el mismo presenta errores en su funcionamiento.	Se implementa un pool genérico pero tiene errores en su funcionamiento.	Se implementa un pool genérico y su funcionamiento es correcto.	10

Principios Solid	Se observan numerosas violaciones de los principios Solid	Se observa un intento de aplicar SOLID, pero de forma confusa o parcial; se observan casos en que las clases siguen teniendo responsabilidades mezcladas.	Se observa que hay una relativa separación de responsabilidades en las clases.	Se observa que hay una correcta separación de responsabilidades en las clases y se cumple el principio open-close.	Se aplican correctamente los principios SOLID, con clases bien separadas, extensibles y con dependencias claras.	10
------------------	---	---	--	--	--	----

Lo anteriormente mencionado servirá para ponderar el 80% del puntaje total.

El restante 20% corresponderá a la defensa oral del proyecto.

Escala de puntuación:

Nota	1	2	3	4	5	6	7	8	9	10
Puntos	1-17	18-38	39-59	60-66	67-72	73-78	79-84	85-89	90-95	96-100